Implementación de la estructura completa de una base de datos (DDL)

Informe

Realizado por: María Sofia Pabón Pabón

Grupo: G101 Bases de Datos

Docente: Oralia Cortes Grajales

Institución Universitaria Pascual Bravo

18 e octubre, del 2026

1.Introducción y elección del SGBD

La comunidad universitaria de la institución universitaria Pascual Bravo requiere una plataforma que impulse la interacción entre todos los estudiantes, facilitando crear perfiles, organizar grupos de estudio, publicación de contenidos, programación de eventos universitarios y encontrar redes de mentoría para el aprendizaje, esto se crea a partir de un modelo conceptual y uno lógico diseñado previamente en tareas anteriores.

El objetivo de el presente informe es documentar la trasformación e implementación del modelo físico relacional funcional en un SGBD

2. Justificación del SGBD seleccionado- PostgreSQL 17

Para dar soporte a la red social se escogió el sistema de gestión de bases de datos PostgreSQL administrado mediante la herramienta grafica pgAdmin 4

PostgreSQL (la elegida)	MySQL	SQLite
Elegida porque considero que la opción más completa y ordenada ya que garantiza que si se cambia algo (como un usuario) lo que esta conectado a el se actualice sin dejar basura, además lo pude entender más fácilmente	Es muy popular para las paginas web, pero tiene deficiencias a comparación de la elegida en cuanto a las reglas y la organización automática de relaciones cuando hay varios usuarios haciendo cosas al tiempo	Es una base de datos muy liviana que guarda todo en un solo archivo es buena para aplicaciones de celular no muy avanzadas, pero para nuestro contexto de red universitaria no nos serviría, no soportaría

Entonces elegí PostgreSQL para la red social pascualina porque considero que es la mas confiable para mantener todo organizado y conectado, asimismo, soporta muchos estudiantes al tiempo y se actualiza automáticamente

3.Descripción de las 15 tablas

estudiante: guarda la cuenta principal del usuario

perfil: contiene ficha publica y datos académicos (relación 1:1 con estudiante)

publicación: almacenas publicaciones compartidas por todos los alumnos

comentario: respuestas asociadas a una publicación

grupo: comunidades o clubes de estudio

evento: talleres, reuniones programadas

interés: catalogo de aficiones

habilidad: destrezas técnicas o académicas

estudiante_habilidad: Tabla intermedia (N:M) que registra las fortalezas de cada alumno

estudiante_interes: Tabla intermedia (N:M) que asocia los gustos del alumno

miembro_grupo: Tabla intermedia (N:M) que gestiona los participantes de cada grupo

asistente_evento: Tabla intermedia (N:M) para confirmaciones de asistencia a eventos

reaccion: Tabla intermedia (N:M) para registrar interacciones (me gusta/apoyo) en publicaciones

seguir: Tabla autorreferenciada (N:M) para la red de seguidores entre estudiantes

mentoria: Tabla autorreferenciada (N:M) para la gestión de tutorías (mentor y aprendiz)

4.Justificación claves primarias y datos

- SERIAL (IDs o códigos): esto se utiliza para ponerle un numero único a cada registro del sistema como (por ejemplo, la cedula que es uno para cada persona), de forma automática, entonces la base de datos lo que hace es sumarle 1 al código cada que se crea algo nuevo, sin necesidad de ponerlo manualmente ni tener el error de repetirlo
- VARCHAR(N) (textos con un limite de tamaño): este es para los datos que contengan texto que mas o menos sabemos cuántas letras máximo puede tener (por ejemplo, los nombres, correos, carrera, etc.) y esto nos ayuda a que la base de datos no ocupe un espacio de memoria innecesariamente
- TEXT (textos sin límite) se usa para campos donde los usuarios pueden escribir tanto como quieran ya que no sabemos de que tamaño será su mensaje, como (por ejemplo, en su biografía y el contenido de su perfil)
- TIMESTAMP (Fechas y horas) Este lo usamos para guardar el momento con precisión de datos relacionados con el tiempo como día, mes, hora,

minutos, de cuando pasa algo en la red como cuando alguien se registra a un programa o evento, entre otras

5.Relaciones e integridad referencial (CASCADE)

Para evitar registros huérfanos y garantizar la consistencia en los datos se implementaron las reglas de integridad referencial en todas sus claves foráneas como:

ON DELETE CASCADE: Aquí, si un estudiante elimina su cuenta el SGBD, borra automáticamente su perfil, sus publicaciones, comentarios, reacciones y sus registros, para que no queden datos sueltos de alguien que ya elimino su perfil

ON UPDATE CASCADE: Significa que si la clave primaria de un registro cambia o se actualiza toda la modificación se hace instantáneamente a todas las otras tablas hijas, por ejemplo, si cambias tu número de identificación de un registro en la tabla principal (tabla padre) la base de datos actualiza las demás tablas donde aparezca (las tablas hijas)

6.Script DDL implementado en SQL y su estructura

El script del código se organiza en tres partes sencillas para su funcionamiento

1. Borra las tablas viejas que puedan existir en la base de datos pare empezar de 0
2. Se arman la estructura de la red social a través de sus 15 tablas y sus casillas en cada tabla, así como sus claves únicas (SERIAL) y conexiones entre tablas con el borrado y la actualización (CASCADE)
3. Por último, guarda un par de datos de ensayo, con un ejemplo de una estudiante llamada Sofia y se hace una consulta rápida para ver que todo funciona correctamente y que se guardo

```
--RED SOCIAL PASCUALINA SCRIPT DDL, SOFIA PABON PABON

--Borramos las tablas viejas para empezar con la base de datos limpia

DROP TABLE IF EXISTS mentoria, seguir, reaccion, asistente_evento, miembro_grupo, estudiante_interes, estudiante_habilidad, interes, habilidad, evento, grupo, comentario, publicacion, perfil, estudiante CASCADE;
```

```
--Procedemos a construir la estructura de la red social (las 15 tablas)
--1. TABLA: estudiante
CREATE TABLE estudiante (
    id_estudiante SERIAL,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    password VARCHAR(255) NOT NULL,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    ultima_actividad TIMESTAMP NOT NULL,
    estado VARCHAR(20) DEFAULT 'activo' NOT NULL,
    CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante),
    CONSTRAINT uk_estudiante_correo UNIQUE (correo)
);

-- 2. TABLA: perfil
CREATE TABLE perfil (
    id_perfil SERIAL,
    id_estudiante INT NOT NULL,
    area_estudio VARCHAR(100),
    biografia TEXT,
    programa VARCHAR(100) NOT NULL,
    semestre INT NOT NULL,
    ciudad VARCHAR(50),
    foto_perfil VARCHAR(255),
    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT
NULL,
```

```
    CONSTRAINT pk_perfil PRIMARY KEY (id_perfil),
    CONSTRAINT uk_perfil_estudiante UNIQUE (id_estudiante),
    CONSTRAINT fk_perfil_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE
);



-- 3. TABLA: publicacion
CREATE TABLE publicacion (
    id_publicacion SERIAL,
    id_estudiante INT NOT NULL,
    contenido TEXT NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT pk_publicacion PRIMARY KEY (id_publicacion),
    CONSTRAINT fk_publicacion_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE
);
-- 4. TABLA: comentario
CREATE TABLE comentario (
    id_comentario SERIAL,
    id_publicacion INT NOT NULL,
    id_estudiante INT NOT NULL,
    comentario TEXT NOT NULL,
    fecha_comentario TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT pk_comentario PRIMARY KEY (id_comentario),
```

```
    CONSTRAINT fk_comentario_publicacion FOREIGN KEY (id_publicacion)
        REFERENCES publicacion(id_publicacion) ON UPDATE CASCADE ON
DELETE CASCADE,
    CONSTRAINT fk_comentario_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE
);

-- 5. TABLA: grupo
CREATE TABLE grupo (
    id_grupo SERIAL,
    creador_id INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    tipo VARCHAR(50) NOT NULL,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    estado VARCHAR(20) DEFAULT 'activo' NOT NULL,
    CONSTRAINT pk_grupo PRIMARY KEY (id_grupo),
    CONSTRAINT fk_grupo_creador FOREIGN KEY (creador_id)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE
);

-- 6. TABLA: evento
CREATE TABLE evento (
    id_evento SERIAL,
    id_grupo INT, -- Opcional: NULL si no pertenece a un grupo
    organizador INT NOT NULL,
```

```
    nombre VARCHAR(150) NOT NULL,
    descripcion TEXT,
    fecha_inicio TIMESTAMP NOT NULL,
    fecha_fin TIMESTAMP NOT NULL,
    lugar VARCHAR(150) NOT NULL,
    modalidad VARCHAR(50) NOT NULL,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT pk_evento PRIMARY KEY (id_evento),
    CONSTRAINT fk_evento_grupo FOREIGN KEY (id_grupo)
        REFERENCES grupo(id_grupo) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT fk_evento_organizador FOREIGN KEY (organizador)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE
);

-- 7. TABLA: interes
CREATE TABLE interes (
    id_interes SERIAL,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    CONSTRAINT pk_interes PRIMARY KEY (id_interes),
    CONSTRAINT uk_interes_nombre UNIQUE (nombre)
);
-- 8. TABLA: habilidad
CREATE TABLE habilidad (
    id_habilidad SERIAL,
    nombre VARCHAR(100) NOT NULL,
```

```
    descripcion TEXT,
    CONSTRAINT pk_habilidad PRIMARY KEY (id_habilidad),
    CONSTRAINT uk_habilidad_nombre UNIQUE (nombre)
);

-- 9. TABLA INTERMEDIA: estudiante_habilidad
CREATE TABLE estudiante_habilidad (
    id_estudiante INT NOT NULL,
    id_habilidad INT NOT NULL,
    nivel VARCHAR(20),
    fecha_agregado TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT pk_estudiante_habilidad PRIMARY KEY (id_estudiante, id_habilidad),
    CONSTRAINT fk_esthab_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_esthab_habilidad FOREIGN KEY (id_habilidad)
        REFERENCES habilidad(id_habilidad) ON UPDATE CASCADE ON DELETE CASCADE
);

-- 10. TABLA INTERMEDIA: estudiante_interes
CREATE TABLE estudiante_interes (
    id_estudiante INT NOT NULL,
    id_interes INT NOT NULL,
    fecha_agregado TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT pk_estudiante_interes PRIMARY KEY (id_estudiante, id_interes),
    CONSTRAINT fk_estint_estudiante FOREIGN KEY (id_estudiante)
```

```
    REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE,
  CONSTRAINT fk_estint_interes FOREIGN KEY (id_interes)
    REFERENCES interes(id_interes) ON UPDATE CASCADE ON DELETE
CASCADE
);

-- 11. TABLA INTERMEDIA: miembro_grupo
CREATE TABLE miembro_grupo (
  id_estudiante INT NOT NULL,
  id_grupo INT NOT NULL,
  fecha_ingreso TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
  rol VARCHAR(30) DEFAULT 'miembro',
  CONSTRAINT pk_miembro_grupo PRIMARY KEY (id_estudiante, id_grupo),
  CONSTRAINT fk_mgrupo_estudiante FOREIGN KEY (id_estudiante)
    REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE,
  CONSTRAINT fk_mgrupo_grupo FOREIGN KEY (id_grupo)
    REFERENCES grupo(id_grupo) ON UPDATE CASCADE ON DELETE
CASCADE
);

-- 12. TABLA INTERMEDIA: asistente_evento
CREATE TABLE asistente_evento (
  id_estudiante INT NOT NULL,
  id_evento INT NOT NULL,
  estado VARCHAR(30) DEFAULT 'confirmado',
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
```

```
    CONSTRAINT pk_asistente_evento PRIMARY KEY (id_estudiante, id_evento),
    CONSTRAINT fk_asist_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE,
    CONSTRAINT fk_asist_evento FOREIGN KEY (id_evento)
        REFERENCES evento(id_evento) ON UPDATE CASCADE ON DELETE
CASCADE
);


-- 13. TABLA INTERMEDIA: reaccion
CREATE TABLE reaccion (
    id_estudiante INT NOT NULL,
    id_publicacion INT NOT NULL,
    tipo_reaccion VARCHAR(30) NOT NULL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT pk_reaccion PRIMARY KEY (id_estudiante, id_publicacion),
    CONSTRAINT fk_reac_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE,
    CONSTRAINT fk_reac_publicacion FOREIGN KEY (id_publicacion)
        REFERENCES publicacion(id_publicacion) ON UPDATE CASCADE ON
DELETE CASCADE
);
-- 14. TABLA INTERMEDIA: seguir
CREATE TABLE seguir (
    id_seguidor INT NOT NULL,
    id_seguido INT NOT NULL,
    fecha_seguimiento TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT
NULL,
```

```
    CONSTRAINT pk_seguir PRIMARY KEY (id_seguidor, id_seguido),
    CONSTRAINT fk_seguir_seguidor FOREIGN KEY (id_seguidor)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE,
    CONSTRAINT fk_seguir_seguido FOREIGN KEY (id_seguido)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE
);
-- 15. TABLA INTERMEDIA: mentoria
CREATE TABLE mentoria (
    id_mentor INT NOT NULL,
    id_aprendiz INT NOT NULL,
    tema VARCHAR(100) NOT NULL,
    estado VARCHAR(30) NOT NULL,
    fecha_mentoria TIMESTAMP NOT NULL,
    CONSTRAINT pk_mentoria PRIMARY KEY (id_mentor, id_aprendiz),
    CONSTRAINT fk_mentoria_mentor FOREIGN KEY (id_mentor)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE,
    CONSTRAINT fk_mentoria_aprendiz FOREIGN KEY (id_aprendiz)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON
DELETE CASCADE
);
--Aqui hacemos una prueba de verificacion, se guardan los datos de ensayo y se
consulta para comprobar que funcione de manera correcta(con ejemplo de mis
datos, falsos)
INSERT INTO estudiante (nombre, apellido, correo, password, ultima_actividad)
VALUES ('Sofia', 'Pabon', 'sofia@pascualbravo.edu.co', 'hash123456', NOW());
INSERT INTO perfil (id_estudiante, programa, semestre, biografia)
```

```
VALUES (1, 'Desarrollo de Software', 3, 'Estudiante de software apasionada por bases de datos.');

INSERT INTO habilidad (nombre, descripcion)

VALUES ('PostgreSQL', 'Base de datos relacional');


INSERT INTO estudiante_habilidad (id_estudiante, id_habilidad, nivel)

VALUES (1, 1, 'Avanzado');

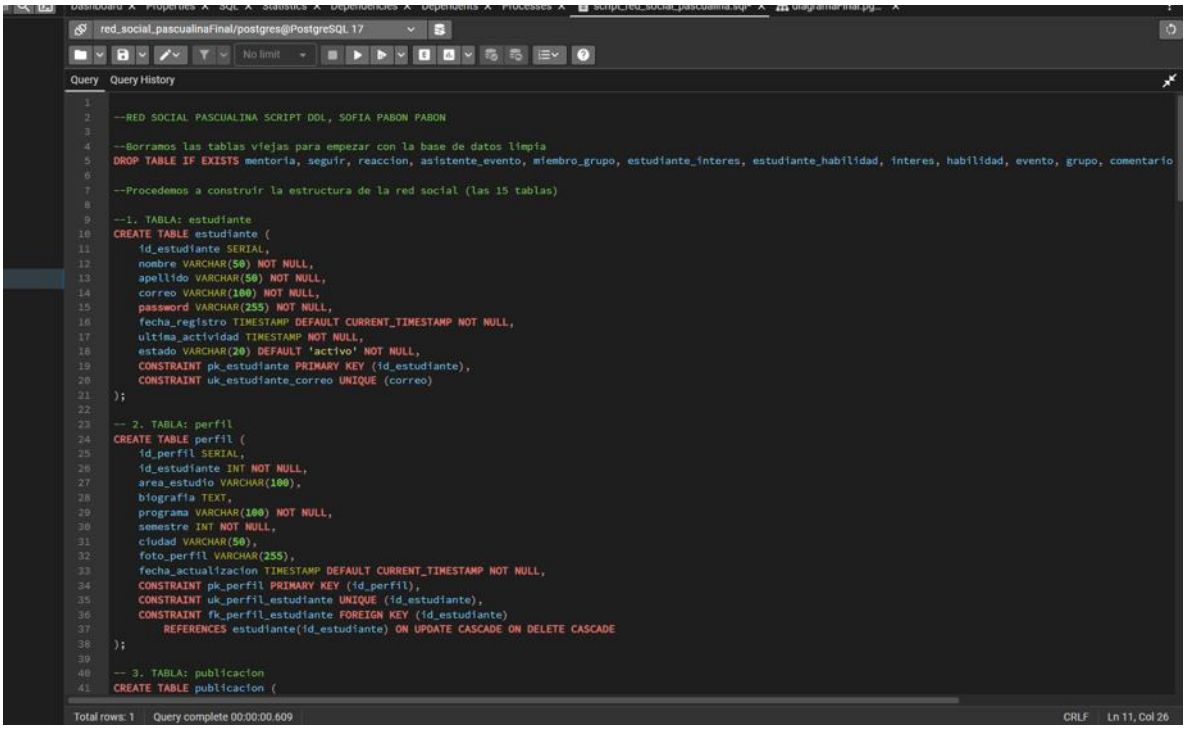

SELECT * FROM estudiante
```

7.Conclusiones y reflexión personal

En este trabajo y en todos los pasos anteriores aprendí lo importante que es pasar de bocetos y diagramas lógicos a un código SQL real aplicado en PostgreSQL, tener toda la parte de las tablas bien pensadas me permitió, que escribir y organizar el script fuera mucho más fácil, mi mayor reto sin duda en actividades pasadas fue entender para que tantas tablas intermedias(las de muchos a muchos) y para que me iban a servir a futuro y con esta practica me ampliaron que son fundamentales para conectar los datos como los estudiantes con sus habilidades, entre las otras que hay en la base de datos.

Para terminar, entendí también que usar las reglas de integridad como (ON DELETE CASCADE) le da mucha seguridad a nuestra base de datos y en mi caso que estoy implementando esto sola tengo que prestar mucha atención a todos los detalles y fijarme en cada cosa puntual, ya que si un usuario borra su cuenta el sistema debe limpiar sin dejar datos por ahí abandonados, tengo que confesar que me daba un poco de inquietud usar pgAdmin ya que era mi primera vez, me preguntaba si era muy complicado y si esa me funcionaba, pero luego de descargarla y ver algunos tutoriales todo fluyo, y lo pude implementar correctamente.

Script-Vista



```
--RED SOCIAL PASCUALINA SCRIPT DDL, SOFIA PABON PABON

--Borramos las tablas viejas para empezar con la base de datos limpia
DROP TABLE IF EXISTS mentoria, seguir, reaccion, asistente_evento, miembro_grupo, estudiante_interes, estudiante_habilidad, interes, habilidad, evento, grupo, comentario

--Procedemos a construir la estructura de la red social (las 15 tablas)

--1. TABLA: estudiante
CREATE TABLE estudiante (
    id_estudiante SERIAL,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    password VARCHAR(255) NOT NULL,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    ultima_actividad TIMESTAMP NOT NULL,
    estado VARCHAR(20) DEFAULT 'activo' NOT NULL,
    CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante),
    CONSTRAINT uk_estudiante_correo UNIQUE (correo)
);

-- 2. TABLA: perfil
CREATE TABLE perfil (
    id_perfil SERIAL,
    id_estudiante INT NOT NULL,
    area_estudio VARCHAR(100),
    biografia TEXT,
    programa VARCHAR(100) NOT NULL,
    semestre INT NOT NULL,
    ciudad VARCHAR(50),
    foto_perfil VARCHAR(255),
    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT pk_perfil PRIMARY KEY (id_perfil),
    CONSTRAINT uk_perfil_estudiante UNIQUE (id_estudiante),
    CONSTRAINT fk_perfil_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE
);

-- 3. TABLA: publicacion
CREATE TABLE publicacion (
```

Diagrama-Vista

